CSCI 567, Fall 2025
Haipeng Luo

Written Assignment #2

Due: Oct 08, 2025, 11:59 pm, PT

Instructions

Total points: 50

Submission: Solutions must be typewritten or neatly handwritten and submitted through gradescope. You can submit multiple times, but only the last submission counts. It is your responsibility to make sure that you submit the right things, and we will *not* consider any regrading requests regarding mistakes in making submissions.

Recall that you have a total of three "late days" for the entire semester, and you can use at most one late day for each written assignment.

Notes on notation:

- Unless stated otherwise, scalars are denoted by small letter in normal font, vectors are denoted by small letters in bold font, and matrices are denoted by capital letters in bold font.
- $\|\cdot\|$ means L2-norm unless specified otherwise, i.e., $\|\cdot\| = \|\cdot\|_2$.

Academic integrity: Our goal is to maintain an optimal learning environment. You can discuss the written assignments at a high level with others, but you should not look at any other's solutions. Trying to find solutions online or from any other sources (including ChatGPT and other similar tools) is prohibited, will result in zero grade and will be reported. To prevent any future plagiarism, uploading any materials from this course to the Internet is also prohibited, and any violations will be reported. Please be considerate and help us help everyone get the best out of this course.

This content is protected and may not be shared, uploaded, or distributed.

Problem 1 Multiclass Perceptron (17 points)

Recall that a linear model for a multiclass classification problem with C classes is parameterized by C weight vectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C \in \mathbb{R}^D$. In the lecture, we derived the multiclass logistic regression by minimizing the multiclass logistic loss. In this problem, you need to derive the multiclass perceptron algorithm in a similar way. Specifically, the multiclass perceptron loss on a training set $(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N) \in \mathbb{R}^D \times [C]$ is defined as

$$F(\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C) = \frac{1}{N} \sum_{n=1}^{N} F_n(\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C), \quad \text{where } F_n(\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C) = \max \left\{ 0, \max_{y \neq y_n} \boldsymbol{w}_y^{\mathrm{T}} \boldsymbol{x}_n - \boldsymbol{w}_{y_n}^{\mathrm{T}} \boldsymbol{x}_n \right\}.$$

1.1 To optimize this loss function, we need to first derive its gradient. Specifically, for each $n \in [N]$ and $c \in [C]$, write down the partial derivative $\frac{\partial F_n}{\partial \boldsymbol{w}_c} \in \mathbb{R}^D$ (and your derivation). For simplicity, you can assume that for any n, $\boldsymbol{w}_1^{\mathrm{T}} \boldsymbol{x}_n, \ldots, \boldsymbol{w}_C^{\mathrm{T}} \boldsymbol{x}_n$ are always C distinct values (so that there is no tie when taking max over them, and consequently no non-differentiable points needed to be considered). (8 points)

1.2 Similarly to the binary case, multiclass perceptron is simply applying SGD with learning rate 1 to minimize the multiclass perceptron loss. Based on this information, fill in the missing details in the repeat-loop of the algorithm below (your solution cannot contain implicit quantities such as $\nabla F_n(\boldsymbol{w})$; instead, write down the exact formula based on your solution from the last question). (4 points)

Algorithm 1: Multiclass Perceptron

Input: A training set $(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)$
Initialization: $\boldsymbol{w}_1 = \cdots = \boldsymbol{w}_C = \boldsymbol{0}$
Repeat:

This content is protected and may not be shared, uploaded, or distributed.

1.3 At this point, you should find that the parameters $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C$ computed by Multiclass Perceptron are always linear combinations of the training points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$, that is, $\boldsymbol{w}_c = \sum_{n=1}^{N} \alpha_{c,n} \boldsymbol{x}_n$ for some coefficient $\alpha_{c,n}$. Just like the binary case discussed in the lecture, this means that one can kernelize multiclass Perceptron as well for any given kernel function $k(\cdot, \cdot)$ (with a corresponding feature mapping $\boldsymbol{\phi}$).

Specifically, fill in the missing details in the repeat-loop of the algorithm below which maintains and updates weights $\alpha_{c,n}$, $\forall c \in [C], n \in [N]$ such that $\boldsymbol{w}_c = \sum_{n=1}^{N} \alpha_{c,n} \boldsymbol{\phi}(\boldsymbol{x}_n)$ is always the same as what one would get by running Algorithm 1 with $\boldsymbol{x}_n$ replaced by $\boldsymbol{\phi}(\boldsymbol{x}_n)$ for all n. No reasoning is required. Keep in mind that in your solution $\boldsymbol{\phi}(\boldsymbol{x}_n)$ should never appear. (5 points)

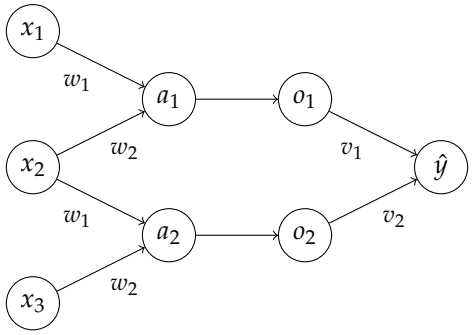
Algorithm 2: Multiclass Perceptron with kernel function $k(\cdot, \cdot)$

Input: A training set $(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)$
Initialize: $\alpha_{c,n} = 0$ for all $c \in [C]$ and $n \in [N]$
Repeat:

This content is protected and may not be shared, uploaded, or distributed.

Problem 2 Backpropagation for CNN (20 points)

Consider the following mini convolutional neural net, where (x_1, x_2, x_3) is the input, followed by a convolution layer with a filter (w_1, w_2), a ReLU layer, and a fully connected layer with weight (v_1, v_2).



More concretely, the computation is specified by

$$\begin{aligned} a_1 &= x_1 w_1 + x_2 w_2 \\ a_2 &= x_2 w_1 + x_3 w_2 \\ o_1 &= \max\{0, a_1\} \\ o_2 &= \max\{0, a_2\} \\ \hat{y} &= o_1 v_1 + o_2 v_2 \end{aligned}$$

For an example $(\boldsymbol{x}, y) \in \mathbb{R}^3 \times \{-1, +1\}$, the logistic loss of the CNN is

$$\ell = \ln(1 + \exp(-y\hat{y})),$$

which is a function of the parameters of the network: w_1, w_2, v_1, v_2.

2.1 First, calculate $\frac{\partial \ell}{\partial \hat{y}}$. Express your final solution using the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$. (2 points)

This content is protected and may not be shared, uploaded, or distributed.

In the next three questions, you are asked to calculate the derivative of ℓ with respect to some variable in terms of its derivative with respect to some other variables, and you can (and should in some cases) express your final solutions in terms of **other neurons/parameters of the network, but not other derivatives that are not mentioned**. In your solutions, **please show the intermediate steps that apply the chain rule.**

2.2 Write down $\frac{\partial \ell}{\partial v_1}$ and $\frac{\partial \ell}{\partial v_2}$ in terms of $\frac{\partial \ell}{\partial \hat{y}}$. (4 points)

2.3 Write down $\frac{\partial \ell}{\partial w_1}$ and $\frac{\partial \ell}{\partial w_2}$ in terms of $\frac{\partial \ell}{\partial a_1}$ and $\frac{\partial \ell}{\partial a_2}$. (4 points)

This content is protected and may not be shared, uploaded, or distributed.

2.4 Write down $\frac{\partial \ell}{\partial a_1}$ and $\frac{\partial \ell}{\partial a_2}$ in terms of $\frac{\partial \ell}{\partial \hat{y}}$. The derivative of the ReLU function is $H(a) = \mathbb{I}[a > 0]$, which you can use directly in your answer. (4 points)

2.5 Combine all derivations above and fill in the missing details of the repeat-loop of the Backpropagation algorithm below that is used to train this mini CNN. Your expression on the gradient needs to be explicit (instead of just $\frac{\partial \ell}{\partial w_1}$ for example). (6 points)

Algorithm 3: Backpropagation for the above mini CNN

1 **Input:** A training set $(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)$, learning rate η
2 **Initialize:** set w_1, w_2, v_1, v_2 randomly
3 **Repeat:**
4 randomly pick an example $(\boldsymbol{x}_n, y_n)$ (the three features of $\boldsymbol{x}_n$ are denoted by x_{n1}, x_{n2}, and x_{n3}.)
5 Forward propagation:
6 Backward propagation:

This content is protected and may not be shared, uploaded, or distributed.

Problem 3 Support Vector Machines (13 points)

Consider a dataset consisting of points in the form of (x, y), where x is a real value, and $y \in \{-1, 1\}$ is the class label. There are only three points $(x_1, y_1) = (-1, -1)$, $(x_2, y_2) = (1, -1)$, and $(x_3, y_3) = (0, 1)$, shown in Figure 1.

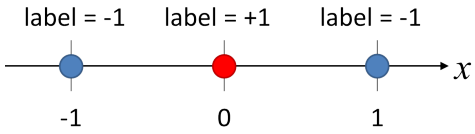


Figure 1: Three data points considered in Problem 1

3.1 Can these three points in their current one-dimensional feature space be perfectly separated with a linear classifier of the form $\text{sgn}(wx + b)$ for some $w, b \in \mathbb{R}$? Why or why not? (2 points)

3.2 Now consider the feature mapping $\boldsymbol{\phi}(x) = [x, x^2]^T$ (convince yourself that the data are now linearly separable in this new feature space). Write down the 3×3 kernel/Gram matrix $\mathbf{K}$ for this dataset. (2 points)

This content is protected and may not be shared, uploaded, or distributed.

3.3 Next, write down the dual formulations of SVM for this dataset in the two-dimensional feature space. Note that when the data is separable, we set the hyperparameter C to be $+\infty$. (You have to plug in the actual data instead of just showing the generic dual formulation.) (2 points)

3.4 Next, solve the dual formulation exactly (note: while this is not generally feasible as discussed in the lecture, the simple form of this dataset makes it possible). (3 points)

This content is protected and may not be shared, uploaded, or distributed.

3.5 Given a test point $x = 2$, what is the prediction of this SVM? Follow the steps below to derive your answer: first, compute the primal solution b^*; then, compute $k(x_n, x)$ for all $n = 1, 2, 3$ and the test point $x = 2$; finally, calculate the prediction. (4 points)

This content is protected and may not be shared, uploaded, or distributed.